

Boundary Layer Simulator (**BL_simulator**)

User Manual and Technical Description

Thermofluids II

Instituto Tecnológico de Buenos Aires (ITBA)

August 24, 2026

Abstract

A modular numerical suite for solving and post-processing laminar boundary layer dynamics (hydrodynamic and thermal) over a isothermal flat plates for both **Forced Convection** and **Natural (Free) Convection** regimes. Developed as academic and computational infrastructure for the Thermofluids curriculum at ITBA. Fully compatible with GNU Octave (≥ 8.4).

1 Overview & System Architecture

The simulator should be executed from within the Thermofluids repository hierarchy; copying **BL_simulator** to an arbitrary directory is not a supported installation method.

BL_simulator is a modular numerical simulator for the analysis of laminar boundary layers over an isothermal flat plates. It provides solutions for both forced-convection and free-convection boundary layers, including the associated hydrodynamic and thermal fields.

The simulator is implemented in GNU Octave and is designed to operate within the Thermofluids software ecosystem.

The application layer is located under **projects/boundary_layer**, while the numerical solvers, physical models, execution infrastructure, and shared definitions are provided by the corresponding modules under **lib/octave**.

For forced convection, the simulator solves the classical self-similar boundary-layer formulation based on the Blasius momentum equation and the Pohlhausen thermal formulation. The energy equation can optionally include viscous dissipation through the Eckert number, $\mathcal{Ec}$.

For free convection along a vertical isothermal flat plate, the simulator uses the Ostrach similarity formulation under the Boussinesq approximation. Viscous dissipation is not included in this formulation because its inclusion breaks the self-similarity of the isothermal-plate solution.

The simulator is intended primarily as academic and computational infrastructure for the study of boundary-layer phenomena in Thermofluids II course dictated at ITBA, but its modular design allows it to be used independently as a numerical tool.

GNU Octave 8.4 or later is the supported execution environment. The code follows MATLAB-compatible syntax and is expected to run under MATLAB, but MATLAB compatibility has not yet been systematically tested.

The simulator depends on shared components of the Thermofluids ecosystem. In particular, it uses the execution infrastructure in `lib/octave/core`, the boundary-layer models and numerical solvers in `lib/octave/phys_bl`, and the shared `+phys` and `+config` modules.

The complete Thermofluids repository is the recommended installation for students and regular users. Users interested only in the Boundary Layer Simulator can instead obtain a minimal checkout containing the simulator and its required shared modules.

Before execution, the required simulation configuration files must be present in: `projects/boundary_layer/config/`. The simulator aborts if the required configuration is missing. The `logs/`, `figures/`, and `outputs/` directories are generated automatically when required.

2 Obtaining the Repository

The Boundary Layer Simulator is designed to operate as an integrated module within the overall **Thermofluids** software hierarchy. To ensure that the execution engine dynamically resolves shared physical constants, numerical solvers, and execution infrastructure, the simulator must reside within a specific directory structure. The repository organizes shared core functionalities under `lib/octave/` while isolating user project applications inside `projects/boundary_layer/`. Below, the required directory tree is presented alongside the Git commands used to retrieve the complete repository.

The repository can be obtained using Git. Git must be installed and available from the system command line. The complete installation and repository setup procedure is documented in the main Thermofluids **README**. For detailed instructions on installing Git, cloning the repository, and preparing the **Thermofluids** environment, see the [main Thermofluids README](#).

```
git clone https://github.com/rviefytes/Thermofluids.git
```

3 Running BL_simulator

`BL_simulator` is the main launcher of the Boundary Layer Simulator. `BL_simulator` must be executed from a directory within the `Termofluids`. The repository root is identified automatically through the `.sandbox` file, so the simulator does not need to be launched from a specific project directory. The `config/` directory is mandatory and must be present in the execution directory. The simulator does not search for configuration files outside this directory. If the `config/` directory is missing, the simulation is aborted.

For example, from the GNU Octave environment pointing to project directory, the simulator can be launched with:

```
BL_simulator
```

in the octave consola.

The launcher initializes the Termofluids environment, loads the required modules, selects the configured case, and executes the corresponding numerical model.

Interactive execution is recommended when graphical output is required. The simulator generates figures during execution. When it is run from the interactive GNU Octave environment, these figures are available in the Octave graphical environment. When the simulator is executed non-interactively from a system command line, the numerical execution and file outputs are still generated, but the interactive figures are not retained in the same way.

4 Configuring a Simulation Case

A simulation case is configured through `run_case.m` and a set of specialized configuration files located in the mandatory `config/` directory. Table 1 shows the configurations files for the `BL_simulator`.

Table 1: Simulation configuration files.

File	Purpose
<code>run_case.m</code>	Selects the problem type and input mode.
<code>set_params_forced.m</code>	Defines physical or dimensionless parameters for forced convection.
<code>set_params_free.m</code>	Defines physical or dimensionless parameters for free convection.
<code>set_flags.m</code>	Controls simulation options.
<code>set_tolerances.m</code>	Defines numerical tolerances.
<code>set_numerical.m</code>	Defines numerical solver options.
<code>set_output_files.m</code>	Defines output file names and related output settings.

4.1 run_case.m: Case Selection

`run_case.m` acts as the case selector. It defines the physical problem and the input mode, while the detailed configuration is delegated to specialized files. This separation keeps the case selection independent from the numerical, physical, output, and execution settings.

```
% Problem type
target_problem = 'FORCED';      % 'FORCED' or 'FREE'
% Input mode
input_mode = 'DIMENSIONLESS';  % 'PHYSICAL' or 'DIMENSIONLESS'
```

target_problem: define the kind of simulation **FORCED** selects the forced-convection formulation based on the Blasius/Pohlhausen similarity solution. **FREE** selects the natural-convection formulation for a vertical isothermal plate based on Ostrach’s similarity solution.

input_mode: define the kind of dimensionality used **PHYSICAL** uses dimensional thermo-physical and geometric parameters. **DIMENSIONLESS** specifies the relevant dimensionless parameters directly. Users normally modify `run_case.m` only to select the problem type and input mode. The remaining simulation parameters should be configured in their corresponding specialized files under `config/`.

4.2 set_params_forced.m: Forced-Convection Parameters

The `set_params_forced.m` script configures the geometric, thermophysical, and flow parameters for forced convection over an isothermal flat plate (Blasius momentum and Pohlhausen thermal formulations).

A key distinction in forced convection is that the free-stream velocity U_∞ is an explicitly known, driven physical boundary condition. Unlike natural convection—where velocity scales arise dynamically from buoyancy forces requiring an equivalent scale ($\mathcal{R}e_{\text{equiv}} = \sqrt{Gr}$)—forced convection directly uses U_∞ to evaluate the physical Reynolds number ($\mathcal{R}e = \rho U_\infty L / \mu$). Listing 1 illustrates the logic and bidirectional parameter mapping implemented in `set_params_forced.m`:

```
% --- 1. GEOMETRY, VELOCITY & THERMODYNAMICS ---
Param.phys.L_plate = 0.5;    % Plate length [m]
Param.phys.U_inf    = 2.0;    % Known free-stream velocity [m/s]
Param.phys.T_w      = 320.0;  % Heated wall temperature [K]
Param.phys.T_inf     = 300.0; % Ambient free-stream temperature [K]
Param.phys.T_film    = 0.5 * (Param.phys.T_w + Param.phys.T_inf); % Reference fluid
                           % properties (evaluated at film temperature T_film)
Param.phys.rho       = 1.177; % Density [kg/m^3]
```

```

Param.phys.k = 0.02624; % Thermal conductivity [W/m*K]
% --- 2. INPUT MODE BRANCHING & ALGEBRAIC BRIDGE ---
if strcmpi(Param.run.mode, 'PHYSICAL')
Param.phys.mu = 1.85e-5; % Dynamic viscosity [Pas]
Param.phys.cp = 1005.0; % Specific heat [J/kgK]

% Forward Mapping (Physical -> Dimensionless)
Param.phys.nu = Param.phys.mu / Param.phys.rho;
Param.phys.Re = (Param.phys.U_inf * Param.phys.L_plate) / Param.phys.nu;
Param.phys.Pr = (Param.phys.mu * Param.phys.cp) / Param.phys.k;
Param.phys.Ec = (Param.phys.U_inf^2) / (Param.phys.cp * abs(Param.phys.T_w - Param.
    phys.T_inf));
elseif strcmpi(Param.run.mode, 'DIMENSIONLESS')
Param.phys.Re = 1e5; % Reynolds number
Param.phys.Pr = 0.7; % Prandtl number
Param.phys.Ec = 0.001; % Eckert number (viscous dissipation)

% Inverse Engineering (Dimensionless -> Physical)
Param.phys.nu = (Param.phys.U_inf * Param.phys.L_plate) / Param.phys.Re;
Param.phys.mu = Param.phys.nu * Param.phys.rho;
Param.phys.cp = (Param.phys.Pr * Param.phys.k) / Param.phys.mu;
end

```

Listing 1: Implementation of `set_params_forced.m`.

4.3 `set_params_free.m`: Free-Convection Parameters

The `set_params_free.m` configuration script specifies geometric parameters, thermophysical fluid properties, and dimensionless governing numbers for natural (buoyancy-driven) convection over a vertical isothermal flat plate. It implements a bidirectional algebraic bridge that seamlessly maps physical inputs ($L, T_w, T_\infty, \rho, \mu, k, c_p$) to dimensionless groups (Gr, Pr, Ra, Ec) and vice versa. It also defines an equivalent Reynolds number ($Re_{equiv} = \sqrt{Gr}$) to unstantiate downstream grid and field mapping routines in the primary simulator engine. Listing 2 demonstrates the dynamic structure and algebraic bridging of `set_params_free.m`:

```

% --- 1. GEOMETRY, TEMPERATURES & THERMODYNAMICS ---
Param.phys.L_plate = 0.5; % Vertical plate height [m]
Param.phys.T_w = 300.5; % Heated wall temperature [K]
Param.phys.T_inf = 300.0; % Unperturbed ambient temperature [K]
Param.phys.T_film = 0.5 * (Param.phys.T_w + Param.phys.T_inf);
C = phys.constants();

```

```

Param.phys.g      = C.G;
Param.phys.deltaT = abs(Param.phys.T_w - Param.phys.T_inf);
Param.phys.beta   = 1 / Param.phys.T_inf; % Ideal gas expansion coef. [1/K]
% --- 2. INPUT MODE BRANCHING & ALGEBRAIC BRIDGE ---
if strcmpi(Param.run.mode, 'PHYSICAL')
Param.phys.mu = 1.963e-5; % Dynamic viscosity [Pas]
Param.phys.cp = 1008.0;   % Specific heat [J/kgK]

% Forward Mapping (Physical -> Dimensionless)
Param.phys.nu = Param.phys.mu / Param.phys.rho;
Param.phys.Pr = (Param.phys.mu * Param.phys.cp) / Param.phys.k;
Param.phys.Gr = (Param.phys.g * Param.phys.beta * Param.phys.deltaT * Param.phys.
    L_plate^3) / (Param.phys.nu^2);
elseif strcmpi(Param.run.mode, 'DIMENSIONLESS')
Param.phys.Gr = 1e8;      % Grashof number (buoyancy driver)
Param.phys.Pr = 0.7;      % Prandtl number
Param.phys.Ec = 0.0;      % Eckert number

% Inverse Engineering (Dimensionless -> Physical)
Param.phys.nu = sqrt((Param.phys.g * Param.phys.beta * Param.phys.deltaT * Param.phys.
    L_plate^3) / Param.phys.Gr);
Param.phys.mu = Param.phys.nu * Param.phys.rho;
Param.phys.cp = (Param.phys.Pr * Param.phys.k) / Param.phys.mu;
end

% --- 3. FLUID DYNAMICS MAPPING EQUIVALENCES ---
Param.phys.Re_equiv = sqrt(Param.phys.Gr);
Param.phys.Re      = Param.phys.Re_equiv; % Required by main spatial mesh routines
Param.phys.Ra      = Param.phys.Gr * Param.phys.Pr;
Param.phys.Ri      = Param.phys.Gr / Param.phys.Re^2; % Yields 1.0 by equivalence

```

Listing 2: Implementation of `set_params_free.m`.

Table 2 summarizes the parameters and variables governed by `set_params_free.m`.

4.4 `set_flags.m`: Simulation Flags

The `set_flags.m` configuration file defines boolean feature toggles (`flg`) that control the execution pipeline. It enables or disables post-processing routines, graphical visualization, file exports, and logging levels without modifying the physical or numerical solvers.

Listing 3 shows a typical configuration of `set_flags.m`:

Table 2: Parameter specifications and physical definitions in `set_params_free.m`.

Parameter Name	Physical Definition & Role
<code>Param.phys.L_plate</code>	Height of the vertical isothermal plate ($L = 0.5$ m).
<code>Param.phys.T_w / T_inf</code>	Wall temperature ($T_w = 300.5$ K) and quiescent fluid temperature ($T_\infty = 300.0$ K).
<code>Param.phys.beta</code>	Volumetric thermal expansion coefficient ($\beta = 1/T_\infty$ for ideal gas approximation).
<code>Param.phys.Gr</code>	Grashof number ($\mathcal{G}r = g\beta\Delta TL^3/\nu^2$), ratio of buoyancy to viscous forces.
<code>Param.phys.Ra</code>	Rayleigh number ($\mathcal{R}a = \mathcal{G}r \cdot \mathcal{P}r$), primary stability parameter for natural convection.
<code>Param.phys.Re_equiv</code>	Equivalent Reynolds number ($\mathcal{R}e_{\text{equiv}} = \sqrt{\mathcal{G}r}$) mapping natural convection scales to unified mesh routines.
<code>Param.phys.Ri</code>	Richardson number ($\mathcal{R}i = \mathcal{G}r/\mathcal{R}e^2 = 1.0$), confirming pure natural convection mode.

```
% --- Export & I/O Control Flags ---
flg.export_figures = true; % Save generated figures to figures/ directory
flg.export_data    = true; % Export numerical profiles (.dat)
flg.export_quiver  = true; % Generate subsampled vector field data for quiver plots

% --- Graphical Visualization Switches ---
flg.plot_similarity = true; % Display similarity profiles (f, f', f'', g, g')
flg.plot_physical   = true; % Display 2D physical field contours (U, V, T)
flg.plot_deltas     = true; % Display boundary-layer thickness profiles

% Export Formats
flg.save.physical_csv = true; % Save physical fields in CSV format
flg.save.vtk          = false; % Generate VTK files for ParaView/Mayavi

% --- Refinate seed for bvp
flg.auto_refine_seed  = true; % refinate seed

%--- Physical audit
flg.audit.phys        = true; % Run audit physics
```

Listing 3: Typical structure of `set_flags.m`.

Table 3 summarizes the main operational flags and their target functions within the simulation pipeline.

Physical conservation audit. When enabled, the simulator evaluates the numerical conservation of mass and energy across sections normal to the plate. This audit is intended

Table 3: Operational switches defined in `set_flags.m`.

Flag Switch	Description & Functionality
<code>flg.export_figures</code>	Controls automatic export of active plots to disk (PDF/PNG format) in <code>figures/</code> .
<code>flg.export_data</code>	Toggles writing tabulated similarity solutions and physical field outputs to <code>outputs/</code> .
<code>flg.export_quiver</code>	Enables subsampling of 2D velocity fields to produce vector CSV files.
<code>flg.plot_similarity</code>	Displays interactive figure windows for dimensionless profiles (f, f', f'', g, g').
<code>flg.plot_physical</code>	Displays interactive figure windows for physical spatial distributions (U, V, T).
<code>flg.plot_deltas</code>	Add boundary layer lines in interactive figure windows for physical spatial distributions.
<code>flg.save.physical_csv</code>	Save physical fields (u, v, T) in csv format.
<code>flg.save.vtk</code>	Save physical fields (u, v, T) in vtk format for Paraview/Mayavi.
<code>flg.auto_refine_seed</code>	Automatically refines the initial seed obtained by the BVP shooting solver.
<code>flg.audit_phys</code>	Enables the numerical audit of mass and energy conservation across sections normal to the plate.

to assess the effects of spatial discretization on the computed solution. In a uniform mesh, larger conservation errors are generally observed near the leading edge, where the boundary-layer gradients are strongest. These deviations provide a useful indication of the spatial resolution required to adequately represent the solution. The audit does not modify the numerical solution; it provides an independent diagnostic of its physical consistency.

4.5 `set_numerical.m`: Numerical Configuration

The `set_numerical.m` script serves as the master configuration orchestrator for numerical dynamic coupling, boundary condition targets, spatial mesh generation, conservation audit modes, and graphics formatting.

Listing 4 displays the essential configuration logic implemented in `set_numerical.m`:

```
% --- 1. PROBLEM-SPECIFIC SOLVER SETUP (DYNAMIC COUPLING) ---
if strcmpi(Param.run.problem_str, 'FREE')
    Param.num.ode_handle = @free_convection_ode;
    Param.num.BC.initial_guess = [0.6; -0.5] * sqrt(2); % [f''(0), g'(0)]
    Param.num.BC.final_targets = [0; 0]; % [f'(inf), g(inf)]
elseif strcmpi(Param.run.problem_str, 'FORCED')
    Param.num.ode_handle = @forced_convection_ode;
    Param.num.BC.initial_guess = [0.3; -0.3] * sqrt(2); % [f''(0), g'(0)]
```

```

Param.num.BC.final_targets = [1; 0];          % [f'(inf), g(inf)]
end
% Known boundary conditions at the wall: f(0)=0, f'(0)=0, g(0)=1
Param.num.BC.known_initial_values = [0; 0; 1];
Param.num.eta_max = 10.0; % Numerical infinity for similarity domain
% --- 2. COMPUTATIONAL GRID CONFIGURATION ---
Param.num.grid.x_start = 1e-4; % Streamwise offset (prevents x=0 singularity)
Param.num.grid.x_end   = Param.phys.L_plate;
Param.num.grid.nx      = 50;    % Nodes in X
Param.num.grid.ny      = 50;    % Nodes in Y
Param.num.grid_s       = 1.08; % Stretching factor (clusters nodes near y=0)
% --- 3. CONSERVATION AUDIT MODE & GRAPHICS ---
Param.num.audit_mode   = 'STRETCHED_FINE';
Param.num.quiver.step  = 2;    % Subsampling stride for vector plots

```

Listing 4: Core structure of `set_numerical.m`.

Table 4 outlines the functional parameters controlled by this configuration script.

Table 4: Numerical and grid parameters defined in `set_numerical.m`.

Parameter Name	Function & Discretization Purpose
<code>Param.num.ode_handle</code>	Function handle pointing dynamically to <code>@forced_convection_ode</code> or <code>@free_convection_ode</code> .
<code>Param.num.BC.initial_guess</code>	Initial estimate seed vector $[f''(0), g'(0)]^T$ supplied to the shooting root-finder.
<code>Param.num.BC.final_targets</code>	Asymptote target vector $[f'(\eta_{\max}), g(\eta_{\max})]^T$ enforced at far-field conditions.
<code>Param.num.eta_max</code>	Upper mathematical domain limit ($\eta_{\max} = 10.0$) truncating boundary layer infinity.
<code>Param.num.grid.x_start</code>	Streamwise spatial offset (10^{-4} m) avoiding singularity at the leading edge ($x = 0$).
<code>Param.num.grid.nx / ny</code>	Number of spatial grid nodes allocated in streamwise (x) and wall-normal (y) directions.
<code>Param.num.grid_s</code>	Mesh stretching ratio ($s = 1.08 > 1$) concentrating grid resolution inside the viscous sublayer near $y = 0$.
<code>Param.num.audit_mode</code>	Scheme used for energy/momentum verification ('STRETCHED_FINE' vs. 'EXACT').
<code>Param.num.quiver.step</code>	Spatial stride subsampling factor to avoid vector overcrowding in quiver visualizations.

4.6 `set_tolerances.m`: Numerical Tolerances

The `set_tolerances.m` script defines a two-stage adaptive precision scheme targeting both the non-linear algebraic root-finder (`fsolve`) and the underlying ODE numerical integra-

tor across the simulation pipeline.

Warning for Users: The parameters in `set_tolerances.m` directly govern solver stability, root-finding convergence, and execution times. They should only be altered by users familiar with numerical analysis. Relaxing these values may yield inaccurate boundary gradients, while excessively tight thresholds can result in non-convergence of `fsolve`.

Listing 5 displays the exact implementation of the nested tolerance structure used by `look_optimal_seed` and `bvp_shooting_solver`:

```
% --- 1. SEED FINDER TOLERANCES (look_optimal_seed) ---
% Loose settings (faster exploration)
Param.num.tolerances.seed_finder.loose.fsolve_tol = 1e-3;
Param.num.tolerances.seed_finder.loose.ode_reltol = 1e-3;
Param.num.tolerances.seed_finder.loose.ode_abstol = 1e-3;
% Tight settings (precise refinement)
Param.num.tolerances.seed_finder.tight.fsolve_tol = 1e-7;
Param.num.tolerances.seed_finder.tight.ode_reltol = 1e-7;
Param.num.tolerances.seed_finder.tight.ode_abstol = 1e-8;
% --- 2. MAIN SOLVER TOLERANCES (bvp_shooting_solver) ---
% Strict tolerances for the high-fidelity final solution
Param.num.tolerances.main_solver.fsolve_tol = 1e-7;
Param.num.tolerances.main_solver.ode_reltol = 1e-7;
Param.num.tolerances.main_solver.ode_abstol = 1e-8;
```

Listing 5: Exact structure of `set_tolerances.m`.

Table 5 details the algorithmic targets governed by this structure.

Table 5: Numerical tolerance specifications defined in `set_tolerances.m`.

Tolerance Group	Function & Algorithmic Target
<code>seed_finder.loose</code>	Coarse tolerances (10^{-3}) for rapid preliminary space sweeping in initial-guess routines.
<code>seed_finder.tight</code>	Fine tolerances (10^{-7} / 10^{-8}) to precisely refine candidate initial condition seeds.
<code>main_solver</code>	Strict thresholds applied during final shooting integration (<code>bvp_shooting_solver</code>).
<code>fsolve_tol</code>	Residual norm termination criteria for the non-linear root-finding algorithm.
<code>ode_reltol</code> / <code>ode_abstol</code>	Local relative and absolute error bounds governing step-size selection in ODE integrators.

4.7 `set_output_files.m`: Output Files

This file defines the output directories and the base names used for simulation results, figures, and logs.

Output filenames include a parameter-dependent suffix identifying the simulation conditions. For forced convection, the suffix includes the problem type, Prandtl number, Eckert number, and Reynolds number. For free convection, Reynolds number is replaced by the Grashof number.

CF → Forced Convection

CL → Free Convection

Pr → Prandtl number

Ec → Eckert number

Re → Reynolds number

Gr → Grashof number

To ensure strict cross-platform filesystem compatibility and prevent shell parsing errors, output file identifiers sanitize floating-point parameter representations by replacing reserved or problematic characters. Decimal points are systematically converted to the character 'p' (e.g., $Pr = 0.7 \rightarrow Pr0p7$), positive scientific exponents 'e+' are shortened to 'e', and negative exponents 'e-' are mapped to 'eneg' (e.g., $Ec = 1.5 \times 10^{-4} \rightarrow Ec1p5eneg4$). Furthermore, floating-point residual values ($|Ec| < 10^{-9}$) are automatically clamped to zero to guarantee clean, deterministic filename suffixes such as:

similarity-solution-data-CF-Pr1p0-Ec0-Re100000.csv

physical-fields-data-CF-Pr1p0-Ec0-Re100000.csv

thickness-hydrodynamic-CF-Pr1p0-Ec0-Re100000.dat

The main output locations are:

Table 6: Output directories defined by `set_output_files.m`.

Directory	Purpose
outputs/	Numerical simulation data.
figures/	Generated figures, when enabled by the simulation configuration.
logs/	Execution logs.

5 Troubleshooting & Common Numerical Issues

Table 7 summarizes common operational issues encountered during execution and their recommended resolution within the simulator.

Table 7: Troubleshooting guide for simulator execution and solver convergence.

Error / Diagnostic Symptom	Probable Cause & Resolution
CORE_ERROR: config/ missing	The launcher was executed outside the repository tree or without the local <code>config/</code> directory. Ensure execution occurs within <code>Termofluids</code> .
<code>fsolve</code> fails to converge	The shooting initial guess is far from the true boundary condition. Enable <code>flg.auto_refine_seed = true</code> in <code>set_flags.m</code> or adjust seed values in <code>set_numerical.m</code> .
Non-asymptotic velocity/temperature profiles	The domain limit $\eta_{\max}$ is too small for the specified Prandtl or Grashof number. Increase <code>Param.num.eta_max</code> in <code>set_numerical.m</code> .

6 Can I run `BL_simulator` without the complete `Termofluids` repository?

Yes. Although cloning the complete `Termofluids` repository is the recommended option for students following the Thermofluids II course, `BL_simulator` can also be obtained using a Git sparse checkout containing only the files required by the simulator.

The required directory structure is:

```
Termofluids/
├── .sandbox
├── lib/
│   ├── octave/
│   │   ├── core/      # Execution and logging infrastructure
│   │   ├── phys_bl/   # Boundary-layer physics and solvers
│   │   ├── +phys/     # Physical constants and definitions
│   │   └── +config/   # Module/path mapping
└── projects/
    ├── boundary_layer/
    │   ├── BL_simulator.m # Simulator Launcher
    │   ├── run_case.m     # Case selector
    │   ├── logs/          # Execution logs
    │   ├── config/        # Required simulation configuration
    │   ├── figures/       # Generated figures
    │   └── outputs/       # Generated data
```

The `figures/`, `logs/`, and `outputs/` directories are created automatically when required.

6.1 Selective checkout

A sparse checkout can be used to obtain only the required parts of the repository. From a command-line terminal:

```
git clone --no-checkout https://github.com/rvieytes/Termofluids.git
cd Termofluids
git sparse-checkout init --cone
git sparse-checkout set \
    lib/octave/core \
    lib/octave/phys_bl \
    lib/octave/+phys \
    lib/octave/+config \
    projects/boundary_layer
git checkout
```

This creates a working copy containing the files required by `BL_simulator`, while leaving the remainder of the `Termofluids` repository outside the working tree.